Complete validation programs for finite normal-pair representations

Human Compression Project

The following are the two complete, unchanged mathematical programs from the accepted proof package. They validate bounded cases; the universal claims and the full exceptional group identifications have written proofs in the two papers. The first program computes both hexagon pairs and the explicit reflection. The second reconstructs the three constructions independently and checks all 25 small colored graphs, seven normal pairs, four completed pairs, and four recursive-family parameters. Neither file is pruned. The archival operational drivers and the separate Lean checks are recorded outside this mathematical-code appendix.

The two hexagon pairs

```
LoadPackage("grape");;
SetAssertionLevel(1);;

P177Hexagon := function(tips, expected)
  local n, edges, j, i, gr, colors, A, C, gen, cls, reflection;
  n:=6+Length(tips);
  edges:=List([0..5],i->Set([i+1,((i+1) mod 6)+1]));
  for j in [1..Length(tips)] do
    i:=tips[j];
    Add(edges,Set([i+1,6+j]));
    Add(edges,Set([((i+1) mod 6)+1,6+j]));
  od;
  gr:=Graph(Group(()),[1..n],OnPoints,
    function(a,b) return Set([a,b]) in edges; end,true);
  colors:=[[1,3,5],[2,4,6],[7..n]];
  A:=AutGroupGraph(gr,colors);
  C:=AutGroupGraph(gr);
  for gen in GeneratorsOfGroup(C) do
    for cls in colors do Assert(1,OnSets(cls,gen) in colors); od;
  od;
  Assert(1,[Size(C),Size(A)]=expected);
  Print("HEXAGON tips=",tips," correlation_order=",Size(C),
    " type_preserving_order=",Size(A)," groups=",
    StructureDescription(C),"/",StructureDescription(A),"\n");
  if Length(tips)=6 then
    reflection:=PermList(Concatenation(
      List([0..5],i->(-i mod 6)+1),
      List([0..5],i->((-i-1) mod 6)+7)));
    Assert(1,reflection in A and Order(reflection)=2);
    Print("TYPE_PRESERVING_REFLECTION ",reflection,"\n");
  fi;
end;;

P177Hexagon([0,2,4],[6,3]);;
P177Hexagon([0..5],[12,6]);;
Print("PHASE177_HEXAGON_CHECK_COMPLETE\n");
QUIT;
```

Bounded independent gadget reconstruction

```
# Reviewer-written bounded checks of the three F010 constructions.
# Universal claims depend on the reviewed written proofs, not these examples.
```

```
LoadPackage("grape");;
SizeScreen([100000,24]);;
R188Check:=function(b,s) if not b then Error(Concatenation("phase188: ",s));fi;end;;
R188Data:=function(colors,edges)
 return rec(colors:=colors,edges:=Set(List(edges,Set)),n:=Length(colors));
end;;
R188Graph:=function(d)
 return Graph(Group(()),[1..d.n],OnPoints,function(a,b)return Set([a,b]) in
  ↪ d.edges;end,true);
end;;
R188Pair:=function(d)
 local palette,classes,n,r,aug,A,C,K;
 n:=d.n;palette:=Set(d.colors);r:=Length(palette);
 classes:=List(palette,c->Filtered([1..n],i->d.colors[i]=c));
 aug:=Graph(Group(()),[1..n+r],OnPoints,function(a,b)
  if a<=n and b<=n then return Set([a,b]) in d.edges;fi;
  if a>n and b>n then return false;fi;
  if a<=n then return d.colors[a]=palette[b-n];fi;
  return d.colors[b]=palette[a-n];
 end,true);
 A:=AutGroupGraph(aug,[[1..n],[n+1..n+r]]);
 C:=Action(A,[1..n],OnPoints);
 R188Check(Size(C)=Size(A),"palette encoding not faithful");
 K:=AutGroupGraph(R188Graph(d),classes);
 R188Check(IsSubgroup(C,K) and IsNormal(C,K),"kernel normality");
 return [C,K];
end;;
R188Degrees:=function(d)
 return List([1..d.n],v->Number(d.edges,e->v in e));
end;;
R188TriangleFree:=function(d)
 return ForAll(d.edges,e->not ForAny([1..d.n],v->Set([v,e[1]]) in d.edges and Set([v,e[2]])
  ↪ in d.edges));
end;;
R188Connected:=function(d)
 local reached,old;
 reached:=[1];old:=[];
 while reached<>old do old:=reached;reached:=Union(old,Concatenation(Filtered(d.edges,e->In
  ↪ tersection(e,old)<>[])));od;
 return Length(reached)=d.n;
end;;
R188CompareRestriction:=function(old,new)
 local p,q,restriction;
 p:=R188Pair(old);q:=R188Pair(new);
 for restriction in [1,2] do
  R188Check(ForAll(GeneratorsOfGroup(q[restriction]),g->OnSets([1..old.n],g)=[1..old.n]),"o
   ↪ ld vertices not invariant");
  R188Check(Size(q[restriction])=Size(p[restriction]) and
   ↪ Action(q[restriction],[1..old.n],OnPoints)=p[restriction],"restriction pair
   ↪ mismatch");
 od;
end;;
R188Complete:=function(d)
 local colors,edges,palette,chambers,e,block,c,r,degrees,result;
 colors:=ShallowCopy(d.colors);edges:=ShallowCopy(d.edges);palette:=Set(colors);r:=Length(p
  ↪ alette);chambers:=[];
 for e in d.edges do
  block:=ShallowCopy(e);
  for c in Difference(palette,d.colors{e}) do Add(colors,c);Add(block,Length(colors));od;
  Append(edges,Combinations(block,2));Add(chambers,Set(block));
 od;
 result:=R188Data(colors,edges);degrees:=R188Degrees(result);
 R188Check(degrees{[1..d.n]}=List(R188Degrees(d),k->k*(r-1)),"old degree formula");
 R188Check(ForAll([d.n+1..result.n],v->degrees[v]=r-1),"new degree formula");
 R188Check(result.n=d.n+(r-2)*Length(d.edges),"completion size");
 return result;
end;;
R188Clean:=function(d)
 local M,W,q,colors,edges,palette,attach,base,j,k,result,deg;
 palette:=Set(d.colors);M:=Maximum(List(palette,c->Number(d.colors,x->x=c)));
 deg:=R188Degrees(d);W:=Filtered([1..d.n],v->deg[v]<=1);q:=Length(d.edges)+Length(W);
 colors:=ShallowCopy(d.colors);edges:=[];
 for attach in Concatenation(d.edges,List(W,v->[v])) do
```

```
  base:=Length(colors)+1;
  for j in [0..2*M+4] do Add(colors,Maximum(palette)+1+(j mod 2));od;
  for j in [0..2*M+3] do Add(edges,[base+j,base+j+1]);od;
  Add(edges,[base+2*M+4,base+2*M-1]);Add(edges,[base+2*M+3,base+2*M]);
  for k in attach do Add(edges,[k,base]);od;
  if Length(attach)=1 and deg[attach[1]]=0 then Add(edges,[attach[1],base+2]);fi;
 od;
 result:=R188Data(colors,edges);
 R188Check(result.n=d.n+q*(2*M+5),"cleaning size");
 R188Check(Minimum(R188Degrees(result))>=2 and R188TriangleFree(result),"cleaning graph
  ↪  hypotheses");
 return result;
end;;

# All partitions of at most three labelled vertices and every compatible graph.
clean_count188:=0;; colors:=[];;
for n in [1..3] do
 for partition in PartitionsSet([1..n]) do
  colors:=List([1..n],v->PositionProperty(partition,c->v in c));
  allowed:=Filtered(Combinations([1..n],2),e->colors[e[1]]<>colors[e[2]]);
  for edges in Combinations(allowed) do
   old188:=R188Data(colors,edges);clean188:=R188Clean(old188);
   R188CompareRestriction(old188,clean188);
   complete188:=R188Complete(clean188);R188CompareRestriction(clean188,complete188);
   clean_count188:=clean_count188+1;
  od;
 od;
od;
R188Check(clean_count188=25,"bounded graph coverage");
Print("CLEANING_AND_COMPLETION_ALL_25_COLORED_GRAPHS_PASS\n");

R188Encode:=function(G,H)
 local els,q,colors,edges,cosets,x,y,k,T,S,i,result,pair,expectedG,expectedH;
 els:=Concatenation([One(G)],Difference(Elements(G),[One(G)]));q:=Length(els);
 cosets:=Set(List(RightCosets(G,H),Set));colors:=List(els,x->PositionProperty(cosets,c->x in
  ↪  c));edges:=[];
 for x in [1..q] do for y in Difference([1..q],[x]) do
  k:=Position(els,els[y]/els[x]);T:=Length(colors)+1;Add(colors,Length(cosets)+1);
  S:=[];for i in [0..k+2] do Add(colors,Length(cosets)+2+(i mod 2));Add(S,Length(colors));od;
  Append(edges,[[x,S[1]],[S[1],y],[x,T],[T,Last(S)],[Last(S),y]]);
  Append(edges,List([1..Length(S)-1],i->[S[i],S[i+1]]));
 od;od;
 result:=R188Data(colors,edges);
 R188Check(result.n=(q^3+9*q^2-8*q)/2 and Minimum(R188Degrees(result))=2 and
  ↪  R188TriangleFree(result) and R188Connected(result),"pair encoding basic invariants");
 R188Check(Size(AutGroupGraph(R188Graph(result)))=q,"uncolored arrow rigidity");
 pair:=R188Pair(result);
 R188Check(Size(pair[1])=Size(G) and Size(pair[2])=Size(H),"normal pair orders");
 expectedG:=Group(List(GeneratorsOfGroup(G),g->PermList(List(els,x->Position(els,x*g^-1)))
  ↪  ));
 expectedH:=Group(Concatenation([()],List(GeneratorsOfGroup(H),g->PermList(List(els,x->Posi
  ↪  tion(els,x*g^-1))))));
 R188Check(Action(pair[1],[1..q],OnPoints)=expectedG and
  ↪  Action(pair[2],[1..q],OnPoints)=expectedH,"exact normal pair right action");
 if q<=3 then R188CompareRestriction(result,R188Complete(result));fi;
 Print("NORMAL_PAIR ",StructureDescription(G)," / ",StructureDescription(H)," vertices
  ↪  ",result.n," completion_checked ",q<=3," PASS\n");
end;;
for G188 in [CyclicGroup(IsPermGroup,2),CyclicGroup(IsPermGroup,3),SymmetricGroup(3)] do
 for H188 in NormalSubgroups(G188) do R188Encode(G188,H188);od;
od;

R188Family:=function(n)
 local old,colors,edges,pairs,pair,j,k,g,map,v,e,result;
 if n=2 then result:=R188Data([2,0,1,2],[[1,2],[2,3],[3,4]]);result.top:=[1,4];return
  ↪  result;fi;
 old:=R188Family(n-1);pairs:=Combinations([1..n],2);
 colors:=Concatenation(List([1..n],i->n),List(pairs,p->n-1));edges:=[];
 for j in [1..Length(pairs)] do for k in pairs[j] do Add(edges,[k,n+j]);od;od;
 for j in [1..n] do
  g:=[1..n];
  if j<>n then k:=First([1..n],i->not i in [j,n]);g[n]:=j;g[j]:=k;g[k]:=n;fi;
  map:=[];
  for k in [1..n-1] do map[old.top[k]]:=n+Position(pairs,Set([g[k],j]));od;
```

```
  for v in Difference([1..old.n],old.top) do
   ↪  Add(colors,old.colors[v]);map[v]:=Length(colors);od;
  Append(edges,List(old.edges,e->map{e}));
 od;
 result:=R188Data(colors,edges);result.top:=[1..n];return result;
end;;
family188:=rec();;
for n in [2..5] do
 family188:=R188Family(n);pair188:=R188Pair(family188);
 R188Check(Size(pair188[1])=Factorial(n) and Size(pair188[2])=Factorial(n)/2,"symmetric pair
  ↪  orders");
 R188Check(Action(pair188[1],family188.top,OnPoints)=SymmetricGroup(n) and
  ↪  Action(pair188[2],family188.top,OnPoints)=AlternatingGroup(n),"faithful natural top
  ↪  action");
 R188Check(R188Connected(family188) and R188TriangleFree(family188),"family connected
  ↪  triangle free");
 if n>=3 then R188Check(Minimum(R188Degrees(family188))>=2,"family degree");fi;
 for i in [0..n-1] do R188Check(Number(family188.colors,c->c=i)=Factorial(n)/(2*Factorial(M
  ↪  aximum(0,i-1))),"family color size");od;
 Print("SYMMETRIC_FAMILY ",n," vertices ",family188.n," color_sizes
  ↪  ",List([0..n],c->Number(family188.colors,k->k=c))," PASS\n");
od;
Print("PHASE188_BOUNDED_GADGET_REVIEW_PASS\n");
QUIT;
```